Capital Group Global Dividend Watch Tool

A self-contained JavaScript widget for exploring dividend payment data from a CSV file. It renders interactive filters, a bar chart, and a data table inside an isolated Shadow DOM component, and supports multi-language display.

Table of contents

Quick start

```
<!DOCTYPE html>
<html>
<head>
  <title>Capital Group Dividend Watch</title>
</head>
<body>
  <h1>Dividend Dashboard</h1>
  <script
    src="https://capitalgroup.fvenn.com/dividendwatch/js/dividend-tool.js"
    data-csv="https://capitalgroup.fvenn.com/dividendwatch/data/Data-
Extract.csv">
  </script>
</body>
</html>
```

The script inserts the widget immediately after itself in the DOM. No build step, no bundler, no other markup required. This tool is centrally hosted at `capitalgroup.fvenn.com` — any page, on any domain, embeds it from that one URL rather than hosting its own copy (this is also why `_headers` grants cross-origin access; see Extending the tool).

If you're running your own separate deployment instead (e.g. local development, or a fork), relative paths work too as long as `index.html` sits alongside `js/`, `css/`, and `data/` — e.g. `<script src="js/dividend-tool.js" data-csv="data/Data-Extract.csv">`.

Script attributes

Attribute	Required	Description
`data-csv`	Yes	Path or URL to the CSV data file
`data-lang`	No	Language code (e.g. `de`, `it`, `es`, `nl`, `zh`). Defaults to English. Accepts a full locale tag (e.g. `de-DE`) and falls back to its base language code.

CSV requirements

The CSV must have a header row with these columns (case-insensitive):

- **`REGION`** — geographic region, e.g. `Europe ex-UK`, `US`
- **`COUNTRY`** — country/territory, e.g. `Germany`, `United States`
- **`SECTOR`** — industry sector, e.g. `Technology`, `Financial`
- **`SUBSECTOR`** — industry subsector, e.g. `Software & IT Services`, `Banks`
- **`YEAR`** — full year, e.g. `2023`
- **`QUARTER`** — format `YYQ#`, e.g. `23Q1`, `10Q3`
- **A dividend column** — any column name containing "dividend" (e.g. `"Dividend, USD"`), numeric

Example

```
REGION,COUNTRY,SECTOR,SUBSECTOR,YEAR,QUARTER,"Dividend, USD"
Canada,Canada,Consumer Cyclical,Auto,2010,10Q3,84533630.4
Europe ex-UK,Germany,Technology,Software & IT Services,2010,10Q2,2300000000
US,United States,Financial,Banks,2023,23Q4,850000000
```

Rows aren't dropped if a value doesn't fit these formats — an unparseable dividend amount is treated as 0, and a quarter value that doesn't match `YYQ#` is kept as-is but won't group correctly into the date range. Malformed data degrades that row rather than stopping the whole tool.

Features

- Multi-dimensional filtering: region → country and sector → subsector (cascading, auto-hides a dropdown when it only has one option)
- Up to three side-by-side datasets (A, B, C) for comparison, each with its own filters
- Yearly or quarterly display, with a custom date range (inclusive)
- Three number formats: USD billions, USD millions, or full USD
- Interactive bar chart (Chart.js) plus a collapsible data table
- Progressive disclosure — nothing renders until "Show Data" is clicked
- Multi-language display via `data-lang` (see below)
- Shadow DOM isolation — the widget's styles can't leak into the host page, and the host page's styles can't leak into the widget

---

Multi-language support

Set `data-lang` on the script tag to display the widget in a supported language:

```
<script
  src="https://capitalgroup.fvenn.com/dividendwatch/js/dividend-tool.js"
  data-csv="https://capitalgroup.fvenn.com/dividendwatch/data/Data-Extract.csv"
  data-lang="de">
</script>
```

Currently shipped: `de` (German), `it` (Italian), `es` (Spanish), `nl` (Dutch), `zh` (Chinese). English needs no language file — omit `data-lang`, or the string content of the tool itself is the English text.

How it works

Each language is a JSON file in `lang/`, named by its code (`lang/de.json`, `lang/it.json`, etc.), mapping the tool's English text to its translation:

```
{
  "All Regions": "Alle Regionen",
  "Dataset": "Datensatz",
  "Show Data": "Daten Anzeigen"
}
```

If a key is missing from the file — or the file itself is missing or fails to load — that string simply falls back to English. A partial or absent translation file degrades gracefully; it never breaks the tool.

Adding a language

1. Copy an existing file, e.g. `lang/de.json`, and translate each value (the keys — the English text — must stay unchanged).
2. Save it as `lang/<code>.json` (e.g. `lang/fr.json` for French).
3. Use `data-lang="<code>"` on the script tag.

Translated data values (region/country/sector/subsector names) are looked up by their **cleaned display label** — the same label that already appears after `labelMappings` normalisation (see below) — not by the raw CSV value. If you add a data-cleanup mapping, add the corresponding translation key for the mapped value, not the raw one.

Customisation

Label mappings (data cleanup)

Some CSV values are inconsistent or abbreviated compared to how they should display. Centralised in `labelMappings` in `js/dividend-tool.js`:

```
const labelMappings = {
  sector: {
    "Media & Telecommunications": "Media & Telcos"
  },
  region: {
    "Pacific Ex China, Hong Kong & Japan": "Pacific Ex China, HK & Japan",
  },
  country: {
    "United Arab Emirates": "UAE",
  },
  subsector: {
    "Auto": "Automotive",
    "Food, drink & Tobacco": "Food, Drink & Tobacco",
    "Household products": "Household Products",
    "Household services": "Household Services",
  },
};
```

This only changes the displayed label — filtering still matches against the raw CSV value.

Styling

All widget styles live in `css/dividend-tool.css`, loaded into the Shadow DOM. Colours and a few layout values are exposed as CSS custom properties on `:host`:

```
:host {
  --background: #ECEFF1;
  --button: #003A66;
  --button-hover: #005F9E;
  --dataset-a: #0074E5;
  --dataset-b: #162252;
  --dataset-c: #008074;
  --chart-text: #333;
  --chart-x-axis: #333;
  --chart-grid: #e8e8e8;
}
```

Override these from the host page by targeting the widget's host element, or edit the file directly.

Number formats

Handled by `fmt()` in `js/dividend-tool.js`. To change the format (e.g. a different currency symbol or decimal convention), edit this function — it's the single place all table and axis values pass through.

Extending the tool

Patterns that work with the tool as it stands today, for integrations beyond a plain `<script>` tag.

Embedding from a React/Vue component

The tool is just a script tag with attributes — no special integration API needed. Inject it into a container element you control, using a ref:

```
function DividendDashboard({ csvUrl, lang }) {
  const containerRef = useRef(null);

  useEffect(() => {
    const script = document.createElement('script');
    script.src = 'https://capitalgroup.fvenn.com/dividendwatch/js/dividend-
tool.js';
    script.dataset.csv = csvUrl;
    if (lang) script.dataset.lang = lang;
    containerRef.current.appendChild(script);
    return () => {
      containerRef.current.innerHTML = ''; // removes the script tag and the
widget it inserted
    };
  }, [csvUrl, lang]);
```

```
  return <div ref={containerRef} />;
}
```

The tool inserts its Shadow DOM host as the next sibling after the script tag, so appending the script into the ref'd `<div>` places the widget inside it correctly.

Shadow DOM isolation means the widget's styles can't clash with the rest of your component tree, and vice versa.

Embedding on a different domain

The `_headers` file's CORS rules already allow the tool's JS, CSS, fonts, language files, and CSV to be fetched cross-origin, so a page on any domain can point a script `src` / `data-csv` at your Cloudflare Pages deployment directly (see Deployment for the URL structure) — the page doesn't need to be served from the same domain as the tool itself.

Replacing the CSV with an API

There's no built-in API mode, but the data-loading step is isolated to one `Papa.parse(csvUrl, {...})` call in `js/dividend-tool.js`. Two ways to swap in a different source, depending on what you control:

- **Your API can return CSV text** — call `Papa.parse(csvText, { header: true, ... })` with the string directly instead of a URL (drop `download: true`).
- **Your API returns JSON** — skip Papa Parse for the initial load and build the `rows` array yourself from the response, using the same shape each row currently has: `{ region, country, sector, subsector, year, quarter, dividend }`. Everything downstream (filtering, aggregation, charting) works against that array regardless of where it came from.

Mapping additional CSV column names

The dividend column is already matched flexibly — any header containing "dividend" (case-insensitive) is normalised to `dividend` (see the column-normalisation loop near the top of the `Papa.parse` `complete` callback). If your CSV uses different names for the other columns (e.g. `territory` instead of `country`), add a similar rule in that same loop rather than renaming columns in the source file.

Architecture

- **Pure vanilla JavaScript**, one file (`js/dividend-tool.js`), no framework, no build step
- **Shadow DOM** — the widget mounts inside a shadow root attached to a `<div>` inserted right after the script tag, isolating its styles and markup from the host page in both directions

- **External dependencies** (loaded from CDN, in parallel): Papa Parse 5.4.1 for CSV parsing, Chart.js 4.4.1 for the bar chart
- **Fonts**: `AvenirLT` is loaded via the `FontFace` API, using a path resolved relative to the script's own URL — this works regardless of what subpath the tool is deployed under, and registers the font for both the widget and the host page (no separate `@font-face` declaration needed anywhere)
- **Data flow**: CSV is fetched and parsed once on load; all filtering, aggregation, and chart/table rendering happens client-side against the in-memory parsed rows

---

Deployment

This repository deploys to Cloudflare Pages automatically: pushing to `main` triggers a build and deploy, no manual upload step. The Cloudflare Pages project is configured with **Root directory** set to `pages-root`, matching this repo's layout:

```
pages-root/
  _headers
  dividendwatch/
    index.html
    js/dividend-tool.js
    css/dividend-tool.css
    lang/*.json
    data/*.csv
    fonts/*.woff2
```

`_headers` must live at `pages-root/_headers` — Cloudflare Pages only reads it from the root of the configured output directory, not from a subfolder. With that root set to `pages-root`, the tool itself is served from `/dividendwatch/`. `_headers` sets CORS (needed so the tool can be embedded on other domains — see Extending the tool), explicit UTF-8 charsets for `.js` / `.json` (needed because non-ASCII characters appear in the tool's own source, e.g. the ▶/▼ collapse icons and translated data), and a short cache lifetime on the CSV data.

A manual zip upload to a Direct Upload project remains possible if ever needed — zip the contents of `pages-root/` (so `_headers` and `dividendwatch/` sit at the zip's top level) and upload as normal.

Browser support

Requires Shadow DOM, `FontFace`, and ES6+ support: Chrome 53+, Firefox 63+, Safari 10.1+, Edge 79+, and their mobile equivalents. No Internet Explorer support.

Troubleshooting

- **"No data-csv attribute on script."** — set the `data-csv` attribute on the script tag.
- **A user-facing error message about loading data/libraries** — the CSV failed to download/parse, or the CDN dependencies failed to load (network issue, ad blocker, CORS). Check the browser console and network tab.
- **A specific piece of UI text is in English when another language is selected** — that string is missing from the relevant `lang/<code>.json` file; add it.
- **A data value (region/country/etc.) isn't translated** — the translation file needs a key matching its *cleaned* display label (see Multi-language support), not the raw CSV value.

Performance

- CSV is parsed once per page load; recommended dataset size is under ~50,000 rows.
- The CSV response is cached for a short period (5 minutes) via `_headers`, since the underlying data is updated periodically — long enough to avoid re-fetching on rapid repeat views, short enough that updates propagate quickly.
- A single Chart.js instance is reused across filter changes (via `.update()`) rather than destroyed and recreated, avoiding both the overhead of reinitialising Chart.js and a "grow from zero" animation replay on every minor change.

Version history

- **v1.0** — Initial release
- **v1.1** — Comparison mode (Dataset B)
- **v1.2** — Responsive design
- **v1.3** — Tooltips and number formatting
- **v1.4** — Range slider and layout improvements
- **v2.0** — Multi-dataset comparison (A, B, C), dropdown-based date range, progressive disclosure
- **v3.0** — Multi-language support, accessibility improvements, performance improvements

---

Licence